EWB CLOUD LABS

VPC Peering Project Report

Setting up and testing private connectivity using AWS VPC peering

Course: AWS Architect Associate

Student Name: Mannat Saini

Batch: AWS Architect Associate Lab Oct 2026

AWS Region: Asia Pacific (Mumbai) / ap-south-1

Submission Date: October 21, 2026

1. Objective and Architecture

1.1 Objective

In this project I connected two separate VPCs (VPC A and VPC B) using VPC peering, so the servers in both can talk to each other over private IPs.

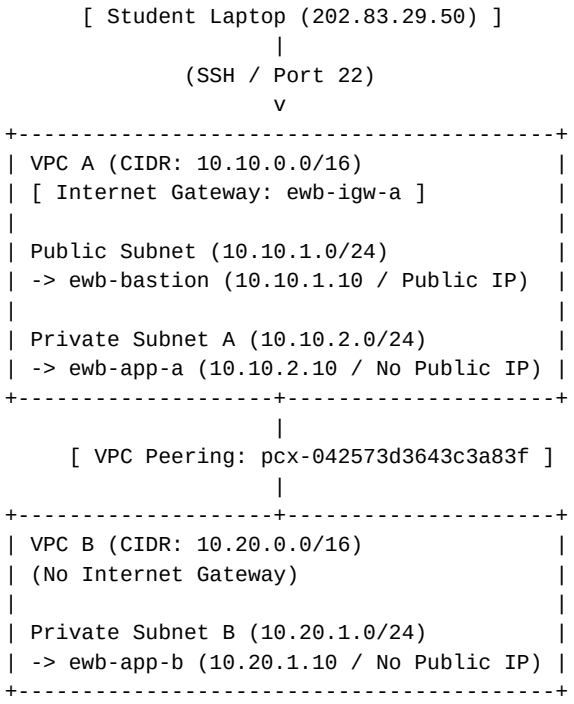
The app servers in both VPCs had to stay private, so no public IPv4 and no direct internet access. To log in to them I used one public bastion host in VPC A with SSH ProxyJump, so the private servers never get exposed to the internet.

1.2 How the traffic flows

Admin access: My laptop connects by SSH to ewb-bastion through the internet gateway in VPC A. From the bastion I can then jump to ewb-app-a or ewb-app-b over the private network.

Traffic between the VPCs: ewb-app-a (10.10.2.10) and ewb-app-b (10.20.1.10) talk to each other directly through the peering connection ewb-a-b-peer. This traffic stays inside AWS and never goes over the internet.

Architecture diagram:



2. Resources and IP Plan

I picked CIDR ranges that don't overlap: 10.10.0.0/16 for VPC A and 10.20.0.0/16 for VPC B. Each subnet is a /24 inside its own VPC range.

Name	Type	Resource ID	CIDR / Private IP	Public IP	Notes
ewb-vpc-a	VPC	vpc-0eafbc5eaea6cd1f4	10.10.0.0/16	-	VPC A (requester)
ewb-a-public	Subnet	subnet-0070774455a4565f0	10.10.1.0/24	-	Bastion host
ewb-a-private	Subnet	subnet-012f5bac8733873fc	10.10.2.0/24	-	App-A
ewb-vpc-b	VPC	vpc-0b5436d63b1fdc9e5	10.20.0.0/16	-	VPC B (accepter)
ewb-b-private	Subnet	(subnet in VPC B)	10.20.1.0/24	-	App-B
ewb-bastion	EC2 t3.micro	i-0d445c79ceb642435	10.10.1.10	15.206.117.187	Jump host
ewb-app-a	EC2 t3.micro	i-0daf97aa56d61eaba	10.10.2.10	None	Private server in VPC A
ewb-app-b	EC2 t3.micro	i-0bc3118c5ca599a00	10.20.1.10	None	Private server in VPC B
ewb-igw-a	Internet gateway	igw-06f277288f79612d3	attached to VPC A	-	Internet for the bastion
ewb-a-b-peer	Peering connection	pcx-042573d3643c3a83f	Active	-	Link between the VPCs
ewb-rt-a-public	Route table	rtb-08897eaea7faaf298	ewb-a-public	-	Bastion subnet
ewb-rt-a-private	Route table	rtb-084a36c262632a83c	ewb-a-private	-	App-A subnet
ewb-lab	Key pair	RSA .pem	-	-	Kept on my laptop

Checkpoint 02: instances running

This is from the EC2 console. All three instances are running. ewb-bastion has a public IPv4 address, but ewb-app-a and ewb-app-b don't have any public IP.

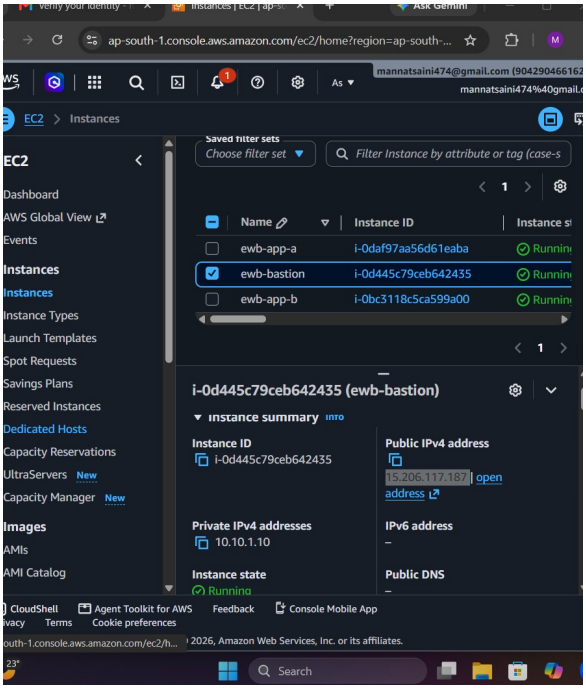


Figure 1: EC2 console showing ewb-bastion (15.206.117.187), ewb-app-a and ewb-app-b running.

3. Subnets and Route Tables

I didn't use the default route tables. I made a separate route table for each subnet and associated them myself.

Route table	Subnet	Destination	Target	Purpose
ewb-rt-a-public	ewb-a-public	10.10.0.0/16	local	VPC A local
		0.0.0.0/0	igw-06f27728...	Internet for bastion
		10.20.0.0/16	pcx-042573d3...	Bastion to VPC B
ewb-rt-a-private	ewb-a-private	10.10.0.0/16	local	VPC A local
		10.20.0.0/16	pcx-042573d3...	App-A to VPC B
ewb-rt-b-private	ewb-b-private	10.20.0.0/16	local	VPC B local
		10.10.0.0/16	pcx-042573d3...	Return route to VPC A

The public subnet also needs the peering route. This is because ProxyJump makes the bastion open its own connection to App-B, so the bastion's subnet has to know how to reach VPC B. The 10.20.0.0/16 route is more specific than 0.0.0.0/0, so that one gets used instead of the internet route. Routes are needed on both sides because the replies need a way back too.

Checkpoint 01, 03 and 04: route and peering screenshots

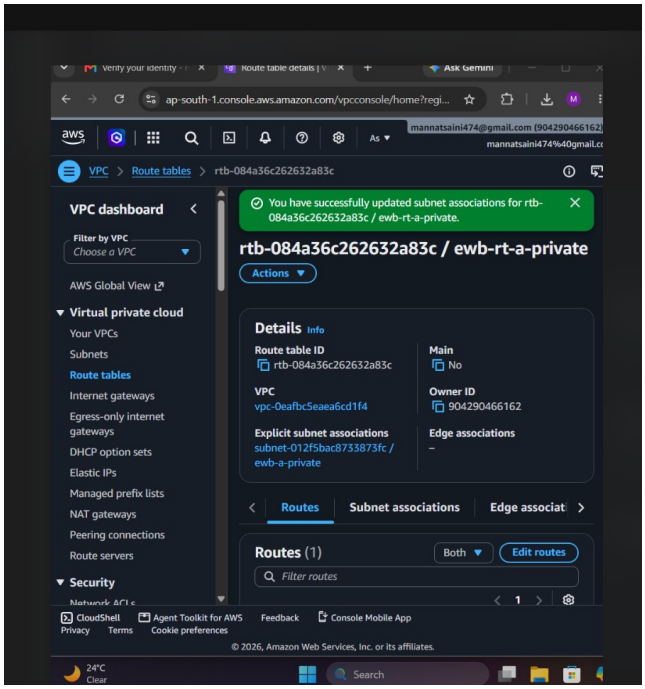


Figure 2: Checkpoint 01 - ewb-rt-a-private explicitly associated with subnet-012f5bac8733873fc (ewb-a-private).

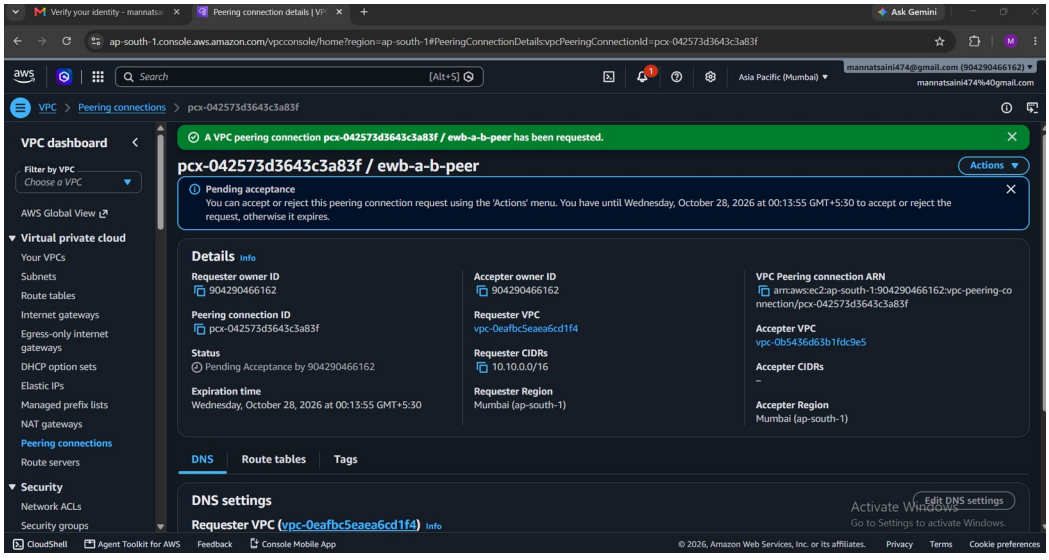


Figure 3: Checkpoint 03 - peering connection ewb-a-b-peer (pcx-042573d3643c3a83f) right after I created the request, from VPC A (vpc-0eafbc5eaea6cd1f4) to VPC B (vpc-0b5436d63b1fdc9e5). Status is Pending acceptance here; I accepted it after this.

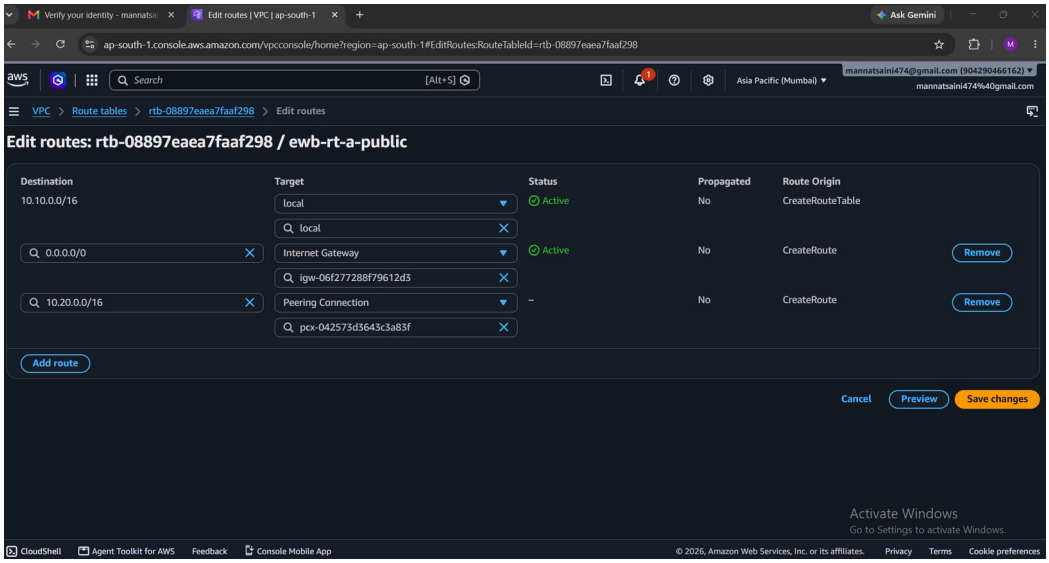


Figure 4: Checkpoint 04 - routes for ewb-rt-a-public: local, 0.0.0.0/0 to igw-06f277288f79612d3, and 10.20.0.0/16 to pcx-042573d3643c3a83f.

4. Security Groups

I used /32 addresses in the rules so only the exact lab hosts are allowed.

- **ewb-sg-bastion (VPC A):** SSH (TCP 22) only from my laptop's public IP, 202.83.29.50/32.
- **ewb-sg-app-a (VPC A):** SSH from the bastion private IP 10.10.1.10/32, and ICMP Echo Request (type 8, code 0) from App-B, 10.20.1.10/32.
- **ewb-sg-app-b (VPC B):** SSH from the bastion private IP 10.10.1.10/32, and ICMP Echo Request from App-A, 10.10.2.10/32.

Security groups are stateful, so replies to allowed traffic go through automatically. I kept the default outbound allow-all rules and the default network ACLs. Nothing is open to 0.0.0.0/0.

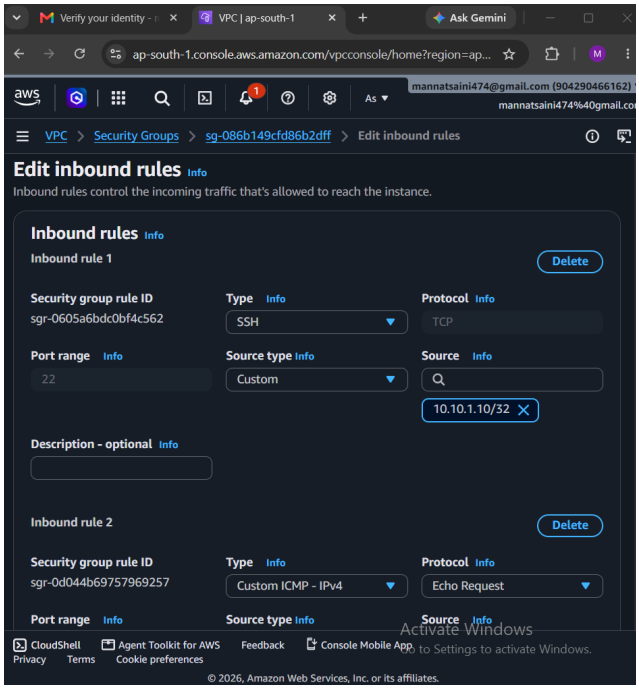


Figure 5: Inbound rules for ewb-sg-app-b - SSH from 10.10.1.10/32 and ICMP Echo Request from 10.10.2.10/32.

5. Testing

Test 01: ping before peering

Command: ping -c 4 10.20.1.10 (from ewb-app-a)

Result: 100% packet loss, no replies.

Before the peering connection and routes were set up, VPC A had no route to 10.20.0.0/16, so the packets just got dropped. This shows the two VPCs were fully isolated at the start.

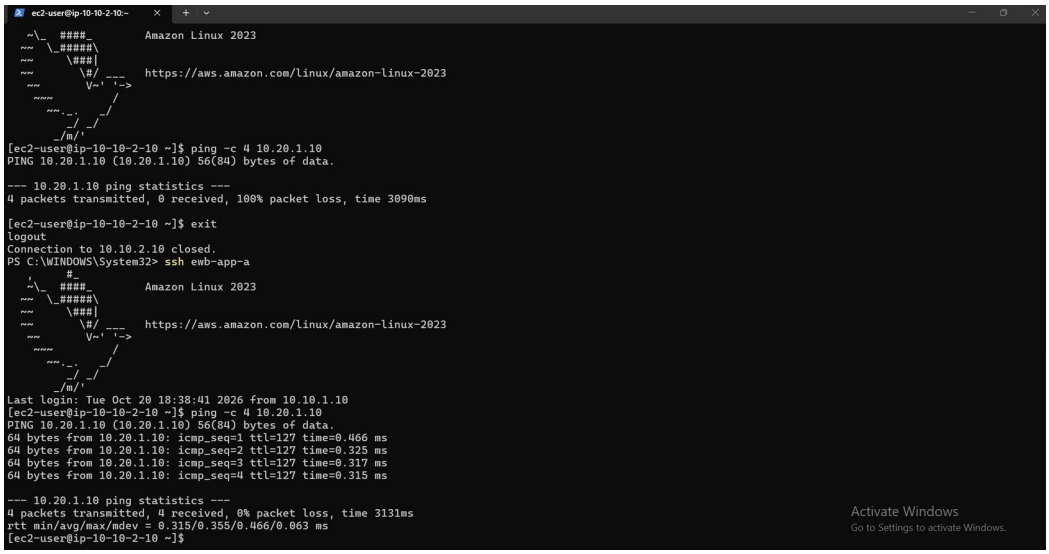


Figure 6: Test 01 - ping from ewb-app-a to 10.20.1.10 before peering, 100% packet loss.

Test 02: App-A to App-B

Command: ping -c 4 10.20.1.10 (from ewb-app-a)

Result: 4 packets sent, 4 received, 0% loss. min/avg/max = 0.315 / 0.355 / 0.466 ms.

The packets went through pcx-042573d3643c3a83f into VPC B, ewb-sg-app-b let them in, and the replies came back over the same peering connection.

Test 03: App-B to App-A

Command: ping -c 4 10.10.2.10 (from inside ewb-app-b)

Result: 4 sent, 4 received, 0% loss. min/avg/max = 0.172 / 0.207 / 0.264 ms.

This works because ewb-rt-b-private has a route for 10.10.0.0/16 back through the peering connection, so routing works both ways.

Test 04: SSH to App-B using ProxyJump

Command: ssh ewb-app-b (from my laptop)

Result: I got a shell prompt: [ec2-user@ip-10-20-1-10 ~]$

My SSH client connected to the bastion (15.206.117.187), and the bastion then connected to 10.20.1.10 through the peering connection. So I could manage App-B without giving it a public IP.

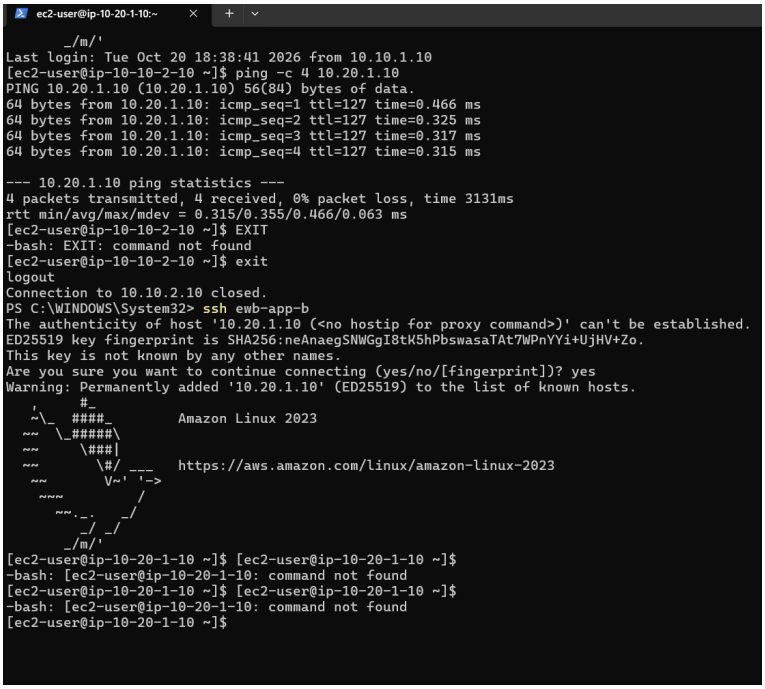


Figure 7: Tests 02 and 04 - ping from App-A to App-B with 0% loss, then SSH to ewb-app-b through the bastion.

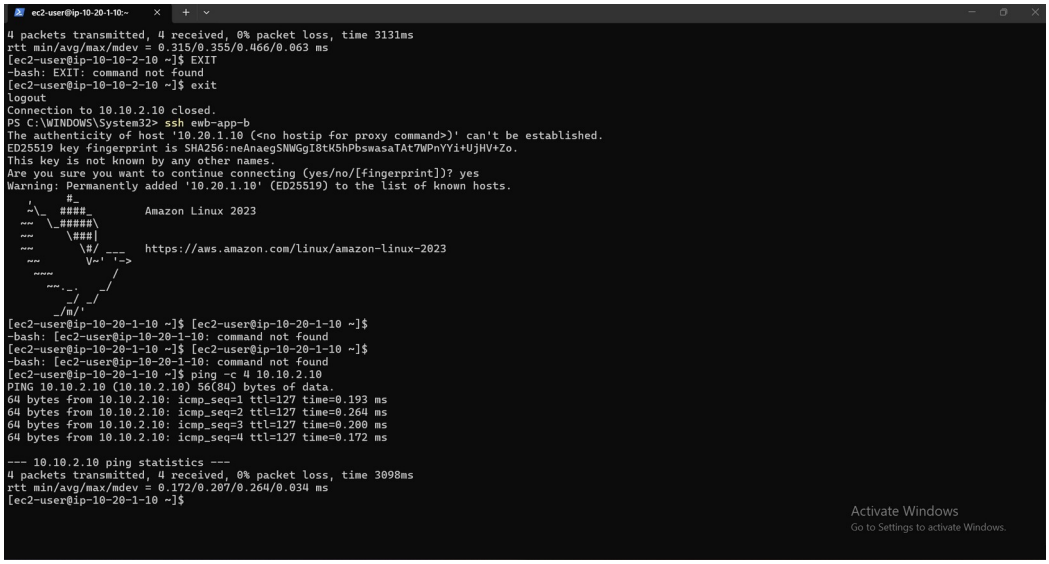


Figure 8: Test 03 - ping from inside ewb-app-b to App-A (10.10.2.10), 0% packet loss.

Test 05: removing and restoring a rule

Failure: I deleted the ICMP Echo Request rule from ewb-sg-app-b and kept the SSH rule. When I ran ping -c 4 10.20.1.10 from App-A again, it showed 100% packet loss. SSH was still connected, so the route was fine and only ICMP was blocked.

Recovery: I added the ICMP rule back (source 10.10.2.10/32) and ran the ping again. It worked straight away, 4 replies and 0% loss.

So even when the routes are correct, the security group still decides if the traffic gets through.

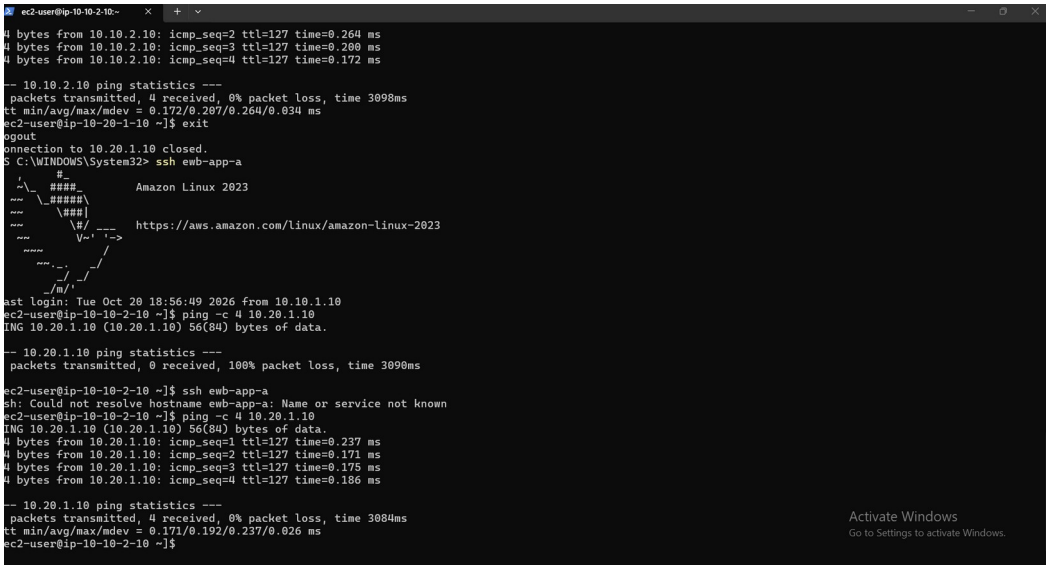


Figure 9: Test 05 - ping from ewb-app-a with 100% loss after the rule was removed, then 0% loss after I added it back.

Isolation check

ewb-app-a and ewb-app-b have no public IPv4 address. Only ewb-bastion has one (15.206.117.187). All my tests used private IPs.

Forward path and return path

Forward: A ping from App-A (10.10.2.10) to 10.20.1.10 matches the 10.20.0.0/16 route in ewb-rt-a-private, which points to pcx-042573d3643c3a83f. The packet goes into VPC B, where ewb-sg-app-b allows the ICMP Echo Request from 10.10.2.10/32.

Return: The reply from 10.20.1.10 is addressed to 10.10.2.10. ewb-rt-b-private has a route for 10.10.0.0/16 to the same peering connection, so the reply goes back into VPC A. Since security groups are stateful, no extra inbound rule is needed for the reply.

Admin: Laptop to bastion over the internet gateway, then the bastion makes a second SSH connection to App-B using the 10.20.0.0/16 peering route in ewb-rt-a-public. App-B sees the bastion private IP 10.10.1.10 as the source.

6. Viva Questions

Q1. Why do we need a return route on both sides of a peering connection?

The route in VPC A only lets packets leave towards VPC B. When App-B sends a reply, it checks its own route table (ewb-rt-b-private). If there's no route for 10.10.0.0/16 pointing to pcx-042573d3643c3a83f, it doesn't know where to send the reply and drops it. So one route isn't enough, both sides need one.

Q2. Why is the SSH source for App-B the bastion's private IP (10.10.1.10/32) and not my laptop IP?

With ProxyJump my laptop only connects to the bastion. Then the bastion opens a second, separate connection from its own private interface (10.10.1.10) to App-B. Since that connection starts inside VPC A, App-B sees 10.10.1.10 as the source, not my laptop IP.

Q3. Why does an Active peering connection not guarantee that ping works?

Active only means the peering link is accepted and connected. It doesn't route any traffic by itself. For ping to work:

- both route tables need a route for the other VPC CIDR pointing to the peering connection,
- the destination security group must allow ICMP Echo Request from the sender, and
- any network ACLs must allow the traffic in both directions.

Q4. Can VPC B use the internet gateway of VPC A through the peering link?

No. VPC peering is not transitive, so VPC B can't use VPC A's internet gateway, and traffic can't come in through a gateway and then jump to another VPC. If VPC B needs internet, it needs its own internet gateway or NAT gateway.

7. Cleanup and What I Learned

7.1 Cleanup

To avoid getting charged I deleted everything in reverse order:

1. Terminated all three EC2 instances (ewb-bastion, ewb-app-a, ewb-app-b) and checked that their EBS volumes were removed.
2. Deleted the peering connection ewb-a-b-peer. Its route table entries were deleted with it.
3. Deleted the security groups I created.
4. Detached and deleted ewb-igw-a.
5. Deleted both VPCs (ewb-vpc-a and ewb-vpc-b). This also removed the subnets and route tables, so only the default VPC is left.

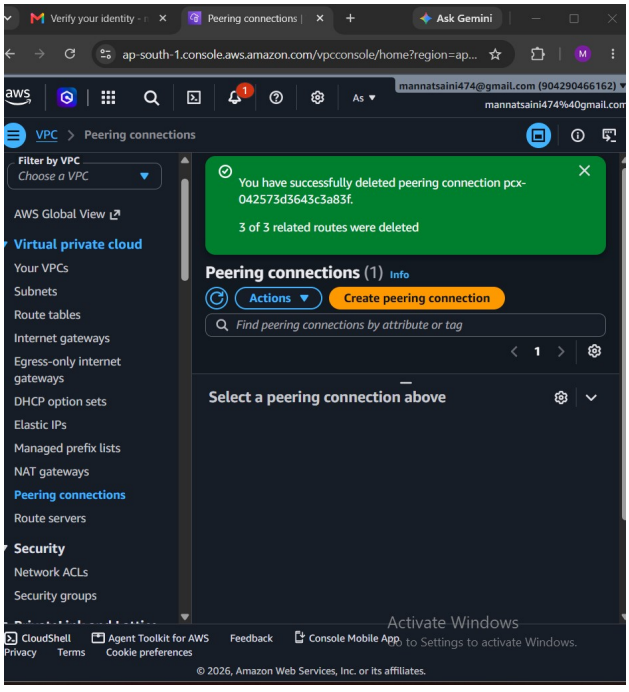


Figure 10: Cleanup - peering connection pcx-042573d3643c3a83f deleted, 3 of 3 related routes removed.

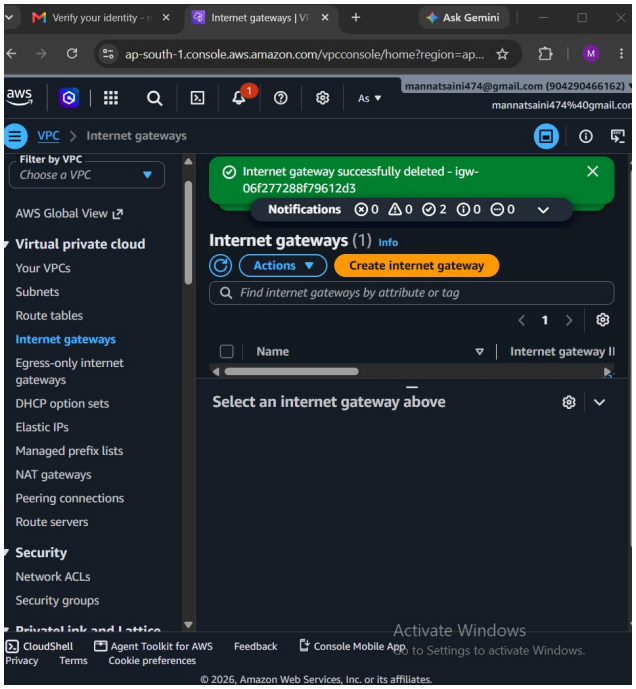


Figure 11: Cleanup - internet gateway igw-06f277288f79612d3 deleted.

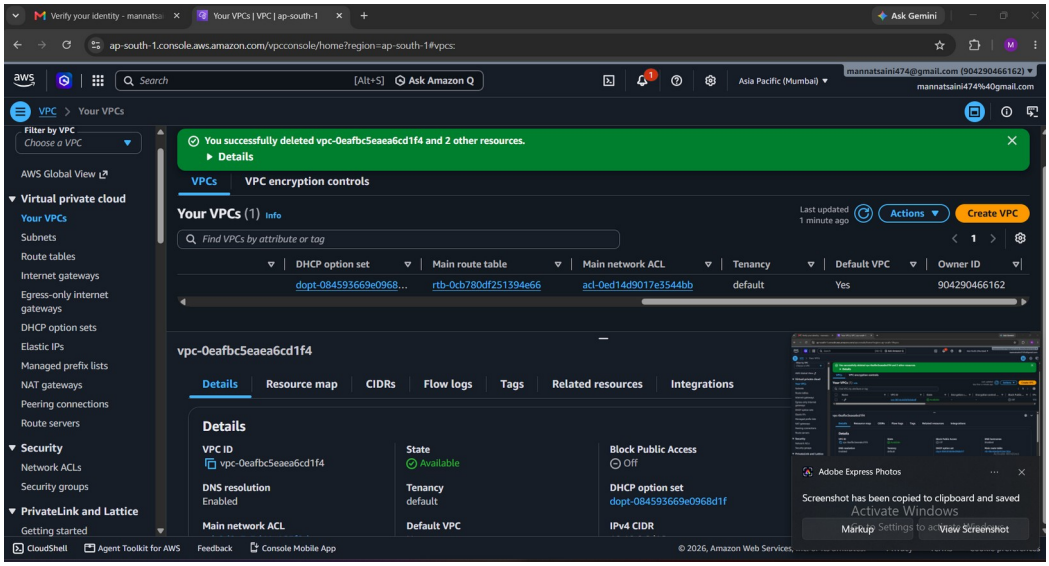


Figure 12: Cleanup - Your VPCs page showing only the default VPC left.

7.2 What I learned

The main thing I learned is that servers can stay completely private and you can still manage them through a bastion host. I also learned that you have to plan non-overlapping CIDRs before building anything, because peering won't work if they overlap. And I understood the difference between routing and security groups better: route tables need a route in both directions, but security groups are stateful so the replies get through automatically.